

DSA Data Structures Fundamentals – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 17 Aug 2026 – 6 Dec 2026 (Days 1–112)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Phase 1–6 parallel — Start Day 8, continue through Month 5.

Prerequisites: JavaScript arrays (PDF #01 §10).

Unlocks: DSA Patterns (08), Advanced DSA (14).

Why this exists (don't skip)

`08-dsa-patterns-complete.pdf` teaches **patterns** but assumes you know what a heap, trie, and graph adjacency list **are**.

`14-advanced-dsa-hard-complete.pdf` is impossible without this foundation.

80 LPA bar: Implement stack, queue, LL, BST, min-heap from scratch in 20 minutes.

Study schedule (parallel with main roadmap)

Week	Sections	Pair with
1-2	§1 Arrays, §2 Strings	LC easy arrays
2-3	§3 Hash Map	LC Two Sum, Anagram
3	§4 Stack, §5 Queue	LC Valid Parentheses
4	§6 Linked List	LC Reverse List
5-7	§7 Tree, §8 BST	LC tree easy
8-12	§9 Graph, §10 Heap	LC BFS/DFS intro
13+	§11 Trie, §12 Union-Find	Month 5 patterns

1. Arrays & Strings

Array operations complexity

Operation	Time
Access by index	$O(1)$
Search (unsorted)	$O(n)$
Insert at end	$O(1)$ amortized
Insert at middle	$O(n)$
Delete	$O(n)$

Implementation patterns

```
// Two pointers on sorted array
function twoSumSorted(nums, target) {
  let l = 0, r = nums.length - 1;
  while (l < r) {
    const sum = nums[l] + nums[r];
    if (sum === target) return [l, r];
    if (sum < target) l++;
    else r--;
  }
  return [-1, -1];
}

// Prefix sum
function prefixSum(nums) {
  const prefix = [0];
  for (const n of nums) prefix.push(prefix.at(-1) + n);
  // sum(i..j) = prefix[j+1] - prefix[i]
  return prefix;
}
```

2. Hash Map / Set

```
// Frequency count pattern
function charFrequency(s) {
  const map = new Map();
  for (const c of s) map.set(c, (map.get(c) || 0) + 1);
  return map;
}

// O(1) lookup - trade space for time
const seen = new Set();
if (seen.has(x)) { /* duplicate */ }
seen.add(x);
```

When to use: Two Sum, anagrams, counting, deduplication.

3. Stack (LIFO)

```
class Stack {  
  constructor() { this.items = []; }  
  push(x) { this.items.push(x); }  
  pop() { return this.items.pop(); }  
  peek() { return this.items.at(-1); }  
  isEmpty() { return this.items.length === 0; }  
}
```

Use cases: Valid parentheses, undo, DFS iterative, monotonic stack.

LC: Valid Parentheses (#20), Min Stack (#155), Daily Temperatures (#739).

4. Queue (FIFO)

```
class Queue {  
  constructor() { this.items = []; }  
  enqueue(x) { this.items.push(x); }  
  dequeue() { return this.items.shift(); } // O(n) – use circular buffer for O(1)  
  front() { return this.items[0]; }  
  isEmpty() { return this.items.length === 0; }  
}
```

Use cases: BFS, task scheduling, rate limiting sliding window.

Better dequeue O(1): Use two stacks or linked list queue.

5. Linked List

```
class ListNode {
  constructor(val = 0, next = null) {
    this.val = val;
    this.next = next;
  }
}

// Reverse linked list – MUST know cold
function reverseList(head) {
  let prev = null, curr = head;
  while (curr) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}

// Fast/slow pointers – cycle detection
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast?.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

Complexity: Access $O(n)$, insert at head $O(1)$.

6. Tree & Binary Search Tree

```
class TreeNode {
  constructor(val = 0, left = null, right = null) {
    this.val = val;
    this.left = left;
    this.right = right;
  }
}

// Traversals – all four MUST know
function inorder(root) { // Left, Root, Right – BST gives sorted
  if (!root) return [];
  return [...inorder(root.left), root.val, ...inorder(root.right)];
}

function bfs(root) {
  if (!root) return [];
  const result = [], queue = [root];
  while (queue.length) {
    const node = queue.shift();
    result.push(node.val);
    if (node.left) queue.push(node.left);
    if (node.right) queue.push(node.right);
  }
  return result;
}
```

BST property: left < root < right. Search/insert $O(\log n)$ balanced, $O(n)$ worst skewed.

7. Graph

```
// Adjacency list (most common)
const graph = {
  A: ["B", "C"],
  B: ["A", "D"],
  C: ["A"],
  D: ["B"],
};

function bfsGraph(start, graph) {
  const visited = new Set([start]);
  const queue = [start];
  while (queue.length) {
    const node = queue.shift();
    for (const neighbor of graph[node] || []) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
  }
  return visited;
}
```

Representations: Adjacency list (sparse graphs), adjacency matrix (dense, $O(1)$ edge lookup).

8. Min-Heap / Priority Queue

```
class MinHeap {
  constructor() { this.heap = []; }

  push(val) {
    this.heap.push(val);
    this.bubbleUp(this.heap.length - 1);
  }

  pop() {
    if (this.heap.length === 1) return this.heap.pop();
    const min = this.heap[0];
    this.heap[0] = this.heap.pop();
    this.bubbleDown(0);
    return min;
  }

  bubbleUp(i) {
    while (i > 0) {
      const parent = Math.floor((i - 1) / 2);
      if (this.heap[i] >= this.heap[parent]) break;
      [this.heap[i], this.heap[parent]] = [this.heap[parent], this.heap[i]];
      i = parent;
    }
  }

  bubbleDown(i) {
    const n = this.heap.length;
    while (true) {
      let smallest = i;
      const l = 2 * i + 1, r = 2 * i + 2;
      if (l < n && this.heap[l] < this.heap[smallest]) smallest = l;
      if (r < n && this.heap[r] < this.heap[smallest]) smallest = r;
      if (smallest === i) break;
      [this.heap[i], this.heap[smallest]] = [this.heap[smallest], this.heap[i]];
      i = smallest;
    }
  }
}
```

Use cases: Top K elements, merge K lists, Dijkstra, median stream.

9. Trie

```
class TrieNode {
  constructor() {
    this.children = {};
    this.isEnd = false;
  }
}

class Trie {
  constructor() { this.root = new TrieNode(); }

  insert(word) {
    let node = this.root;
    for (const c of word) {
      if (!node.children[c]) node.children[c] = new TrieNode();
      node = node.children[c];
    }
    node.isEnd = true;
  }

  search(word) {
    let node = this.root;
    for (const c of word) {
      if (!node.children[c]) return false;
      node = node.children[c];
    }
    return node.isEnd;
  }
}
```

Use cases: Autocomplete, word search, prefix matching.

10. Union-Find (Disjoint Set)

```
class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);
    this.rank = Array(n).fill(0);
  }

  find(x) {
    if (this.parent[x] !== x) this.parent[x] = this.find(this.parent[x]);
    return this.parent[x];
  }

  union(x, y) {
    const px = this.find(x), py = this.find(y);
    if (px === py) return false;
    if (this.rank[px] < this.rank[py]) this.parent[px] = py;
    else if (this.rank[px] > this.rank[py]) this.parent[py] = px;
    else { this.parent[py] = px; this.rank[px]++; }
    return true;
  }
}
```

Use cases: Number of islands, cycle detection, Kruskal MST.

11. Complexity summary

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Hash Map	–	$O(1)^*$	$O(1)^*$	$O(1)^*$
Stack/Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Linked List	$O(n)$	$O(n)$	$O(1)$ head	$O(1)$ head
BST balanced	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	–	$O(n)$	$O(\log n)$	$O(\log n)$
Trie	–	$O(m)$	$O(m)$	$O(m)$

*m = word length, *average case for hash

12. Gate to DSA Patterns (PDF #12)

Before Month 5 patterns intensive:

- ☐ Implement stack, queue, LL reverse from memory
- ☐ BFS tree and graph
- ☐ Min-heap push/pop
- ☐ Explain BFS vs DFS use cases
- ☐ 60+ LC easy/medium done

13. Interview Q&A

Q: Array vs Linked List — when to use which?

A: Arrays: $O(1)$ random access, cache-friendly, fixed or dynamic size — use for index-based access, sorting, two pointers. Linked lists: $O(1)$ insert/delete at known position, no reallocation — use for frequent insertions at head, LRU cache implementation, polynomial arithmetic. Arrays dominate in practice due to cache locality.

Q: How does a hash map work internally?

A: Hash function maps key to bucket index. Collisions handled by chaining (linked list per bucket) or open addressing (probe next slot). JavaScript Map uses hash table with insertion order. Average $O(1)$ get/set; worst $O(n)$ if all keys collide. Load factor triggers resize/rehash.

Q: BFS vs DFS — when to use which?

A: BFS: shortest path in unweighted graph, level-order traversal, nearest neighbor — uses queue, $O(V+E)$. DFS: detect cycles, topological sort, path existence, maze solving — uses stack/recursion, $O(V+E)$. BFS uses more memory for wide graphs; DFS can stack overflow on deep graphs.

Where to practice

- LeetCode Explore: Data Structure cards
- Visualize: visualgo.net
- Implement each structure in `mern-mastery/dsa-structures/` folder

DSA Data Structures Interview Q&A Bank – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

50 interview questions with full detailed answers for arrays, linked lists, stacks, queues, trees, heaps, graphs, and tries.

Target companies: Google, Amazon, Flipkart, Uber, Microsoft, Meta.

How to use: Pick 2 questions daily. Read the question, answer aloud in 2–3 minutes, then check the full answer.

Q1: What is an array and what are its core operations' time complexities?

Answer: An array is a contiguous block of memory storing elements at indices 0 to $n-1$. Access by index is $O(1)$ because $\text{address} = \text{base} + \text{index} \times \text{element size}$. Search unsorted array is $O(n)$. Insert or delete at end is $O(1)$ amortized for dynamic arrays; at middle requires shifting — $O(n)$. Arrays excel at random access and cache locality (CPU prefetch). Most interview problems start with arrays — master index arithmetic and bounds.

Q2: What is a dynamic array and how does resizing work?

Answer: Dynamic arrays (JavaScript Array, Python list, Java ArrayList) grow when full — typically double capacity, copy elements to new block — amortized $O(1)$ append. Occasional $O(n)$ resize. Capacity vs length: allocated slots vs used slots. Interview tip: mention amortized analysis when asked why append is $O(1)$. Pre-allocate when size known to avoid repeated copies.

Q3: When should you use an array vs a linked list?

Answer: Use arrays for random access, sorting, binary search, two-pointer techniques, and when memory locality matters. Use linked lists when you need $O(1)$ insert/delete at a known node (with pointer), no reallocation, or implementing LRU cache with hash map + doubly linked list. Arrays dominate in practice due to cache performance — linked lists win in specific structural problems, not general storage.

Q4: What is a singly linked list and how do you reverse it?

Answer: Singly linked list nodes hold `value` and `next` pointer. Reverse iteratively: maintain `prev`, `curr`, `next`; set `curr.next = prev`, advance pointers — $O(n)$ time, $O(1)$ space. Recursive reverse: reverse rest of list, point next node back — $O(n)$ space for call stack. Classic Amazon and Microsoft phone screen question. Always clarify if singly or doubly linked.

Q5: What is a doubly linked list and why use it?

Answer: Doubly linked lists add `prev` pointer — traverse backward, delete node in $O(1)$ if you have pointer to node (no need to find predecessor). Used in LRU cache (move to front on access), browser history, text editor cursors. Cost: extra memory per node and more pointer updates on insert. Interview: LRU Cache is the flagship doubly linked list + hash map problem.

Q6: How do you detect a cycle in a linked list?

Answer: Floyd's tortoise and hare: slow pointer moves 1 step, fast 2 steps. If they meet, cycle exists. To find cycle start: reset slow to head, move both 1 step until they meet at entrance — $O(n)$ time, $O(1)$ space. Alternative: hash set of visited nodes — $O(n)$ space. Explain why fast-slow works — distance modulo cycle length.

Q7: What is a stack and what are its operations?

Answer: Stack is Last-In-First-Out (LIFO): push (add top), pop (remove top), peek (view top). All $O(1)$ with array or linked list backend. Use for matching parentheses, undo operations, depth-first search (DFS) iterative, evaluating expressions, monotonic stack problems. JavaScript: use array with `push` / `pop` — do not use `shift` ($O(n)$).

Q8: What is a queue and what are its operations?

Answer: Queue is First-In-First-Out (FIFO): enqueue (add rear), dequeue (remove front). $O(1)$ with linked list or circular buffer. Use for breadth-first search (BFS), task scheduling, rate limiting buffers. JavaScript lacks efficient queue on array — use linked list or track head index. Circular queue reuses array slots with front/rear pointers modulo capacity.

Q9: What is a deque (double-ended queue)?

Answer: Deque allows insert and delete at both front and rear in $O(1)$. Implements queue, stack, or sliding window max (with monotonic deque). JavaScript has no built-in deque — simulate with doubly linked list or use `deque` in Python interviews. Sliding window maximum is the classic deque interview problem at Google.

Q10: How can you implement a queue using two stacks?

Answer: Use stack `in` for enqueue (push), stack `out` for dequeue. On dequeue, if `out` empty, pop all from `in` pushing to `out`, then pop `out`. Amortized $O(1)$ per operation. Demonstrates understanding of converting LIFO to FIFO. Follow-up: thread safety and Amortized analysis explanation.

Q11: What is a binary tree?

Answer: Binary tree: each node has at most two children (left, right). Root has no parent. Leaf has no children. Height of node = edges to deepest leaf; depth = edges from root. Full binary tree: every node has 0 or 2 children. Complete binary tree: filled level by level left to right — basis for heap array storage. Binary trees underpin Binary Search Trees and heaps.

Q12: What is a Binary Search Tree (BST) and its complexity?

Answer: BST property: left subtree values $<$ node $<$ right subtree values. Search, insert, delete average $O(\log n)$ when balanced; worst $O(n)$ if skewed (sorted insert). In-order traversal yields sorted order. Self-balancing variants (AVL, Red-Black) guarantee $O(\log n)$. Always mention balance in interviews — unbalanced BST is a linked list in disguise.

Q13: Explain tree traversals: in-order, pre-order, post-order, level-order.

Answer: In-order (left, node, right): sorted order for BST. Pre-order (node, left, right): copy tree, prefix expressions. Post-order (left, right, node): delete tree, postfix expressions. Level-order: BFS with queue, level by level — shortest path in unweighted tree. Know recursive and iterative versions. Flipkart often asks level-order zigzag variant.

Q14: What is the difference between height and depth of a tree?

Answer: Depth of node = number of edges from root to node (root depth 0). Height of node = edges to farthest leaf below it. Tree height = root height. Balanced tree height $\approx \log n$. Skewed tree height = $n-1$. These definitions matter for recursion stack depth and $O(\log n)$ vs $O(n)$ claims.

Q15: What is a heap and what is the heap property?

Answer: Heap is complete binary tree with heap property: min-heap — parent $\leq$ children (root is minimum); max-heap — parent $\geq$ children. Stored as array: parent at i , children at $2i+1$ and $2i+2$. Insert and extract-min are $O(\log n)$ via bubble up/down. Use for priority queues, Top K elements, Dijkstra's algorithm, merge K sorted lists.

Q16: Compare min-heap vs max-heap use cases.

Answer: Min-heap: get smallest element — Dijkstra, scheduling earliest deadline, kth smallest stream. Max-heap: get largest — Top K frequent, median from two heaps (max-heap lower half, min-heap upper half). JavaScript: implement manually or use library; Python `heapq` is min-heap only — negate values for max-heap.

Q17: What are heap insert and extract operations?

Answer: Insert: append to end of array, bubble up comparing with parent until heap property restored — $O(\log n)$. Extract-min: swap root with last element, remove last, bubble down choosing smaller child — $O(\log n)$. Build heap from array in $O(n)$ with bottom-up heapify — know this beats n insertions for initialization.

Q18: What is a priority queue and how is it implemented?

Answer: Priority queue supports insert with priority and extract highest (or lowest) priority element. Binary heap gives $O(\log n)$ insert and extract. Used in task schedulers, event simulation, A* search. Unlike FIFO queue, order depends on priority not arrival time. Fibonacci heap improves Dijkstra theoretically but binary heap is standard in interviews.

Q19: How do you represent a graph?

Answer: Adjacency list: array of lists — for each vertex, list neighbors. Space $O(V+E)$, best for sparse graphs. Adjacency matrix: $V \times V$ boolean/weight matrix — $O(1)$ edge lookup, $O(V^2)$ space, good for dense graphs. Edge list: list of pairs — simple for Kruskal's algorithm. State representation choice in every graph interview.

Q20: What is the difference between directed and undirected graphs?

Answer: Undirected: edge $A-B$ means travel both ways — friendships (mutual). Directed: edge $A \rightarrow B$ one way — Twitter follow, dependency graph. Degree: undirected counts both ends; directed splits in-degree and out-degree. Cycle detection differs — directed needs three-color DFS or topological sort; undirected uses Union-Find or DFS parent tracking.

Q21: What is a weighted graph?

Answer: Edges carry weights (cost, distance, time). Shortest path uses Dijkstra (non-negative weights), Bellman-Ford (negative edges), Floyd-Warshall (all pairs small V). Unweighted shortest path uses BFS. Uber route and network latency problems map to weighted graphs — always clarify weights non-negative before saying Dijkstra.

Q22: When do you use adjacency list vs adjacency matrix?

Answer: List: sparse graph (social network, road map) — memory efficient, iterate neighbors fast. Matrix: dense graph or need $O(1)$ `isEdge(u,v)` — Floyd-Warshall, small $V \leq 500$. For $V=10^6$ and $E=10^7$, matrix is impossible — list mandatory. Interview: ask V and E before picking.

Q23: What is a trie (prefix tree)?

Answer: Trie stores strings character by character in tree nodes; each path from root spells a prefix. Search, insert, delete $O(m)$ where m = word length. Shared prefixes save space vs storing full strings separately. Used for autocomplete, spell check, IP routing tables, word search on board. Each node map/array of 26 children for lowercase English.

Q24: What problems are tries especially good for?

Answer: Prefix search: all words starting with 'app' — autocomplete. Exact match faster than hash for many short strings with shared prefixes. XOR maximum pair (bit trie). Less ideal when alphabet huge or strings very long with no shared prefixes — hash map may win. Google search suggestion is trie-class problem.

Q25: How does a hash map work internally?

Answer: Hash function maps key to bucket index. Store key-value in bucket. Collisions: chaining (linked list per bucket) or open addressing (probe next slot). JavaScript Map maintains insertion order with $O(1)$ average get/set. Load factor triggers resize and rehash. Worst case $O(n)$ if all keys collide — mention in security context (hash flooding).

Q26: What is collision resolution by chaining?

Answer: Each bucket holds a list of entries whose keys hash to same index. Lookup: hash, traverse list comparing keys. Simple, handles load > 1 with linked lists. Java HashMap before Java 8 used chaining; now treeify long buckets. Average $O(1)$ with good hash and load factor ~ 0.75 .

Q27: What is a hash set and when to use it vs hash map?

Answer: Hash set stores keys only — $O(1)$ average contains check. Use for deduplication, cycle detection (visited set in BFS/DFS), two-sum complement lookup when you only need existence. Hash map when you need associated value (frequency count, index mapping). Both $O(n)$ space trade for $O(1)$ time.

Q28: What is Union-Find (Disjoint Set Union)?

Answer: Union-Find tracks partitioned sets with `find(x)` (representative of x's set) and `union(x,y)` (merge sets). Path compression and union by rank give nearly $O(1)$ amortized — inverse Ackermann. Use for Kruskal MST, connected components, dynamic connectivity, detecting cycle in undirected graph. Cleaner than DFS when only connectivity matters.

Q29: How would you design an LRU cache?

Answer: Least Recently Used cache with $O(1)$ get and put: hash map (key $\rightarrow$ node pointer) + doubly linked list (order by recency). On get: move node to head. On put: add to head, evict tail if over capacity. Capacity bounded memory. Classic Meta and Amazon Low-Level Design (LLD) question — know exact data structures and complexities.

Q30: What is a circular linked list?

Answer: Last node points back to first instead of null. Used in round-robin scheduling, Josephus problem, buffer rings. Detect cycle variant: any node's next eventually returns. Breaking cycle carefully on delete. Sometimes used to simulate infinite list with fixed memory.

Q31: What is a skip list?

Answer: Skip list: layered linked lists with express lanes — search $O(\log n)$ average probabilistically. Simpler to implement than balanced BST; used in Redis sorted set internals. Each node randomly promoted to upper levels. Alternative when red-black tree rotation logic is too heavy — mention conceptually in advanced interviews.

Q32: What is the difference between a B-tree and a binary search tree?

Answer: B-tree nodes hold many keys and children — optimized for disk pages (database indexes). BST is binary in memory. B-tree height stays low for millions of keys — fewer disk reads. Interview link: MongoDB and PostgreSQL indexes use B-tree variants; in-memory algorithm problems use BST or hash.

Q33: How do you find in-order successor in a BST?

Answer: If node has right subtree: successor is leftmost of right subtree. Else: walk up until current is left child — parent is successor. $O(h)$ time. Used in BST delete when replacing with successor. Shows pointer manipulation understanding beyond textbook traversals.

Q34: What is level-order traversal and what data structure does it use?

Answer: Level-order (BFS on tree): visit nodes row by row using a queue. Process node, enqueue left then right children. Used for printing levels, finding depth, serializing tree. Zigzag variant reverses every other level with deque or reverse slice. $O(n)$ time, $O(w)$ space where w = max width.

Q35: What is a complete vs full vs perfect binary tree?

Answer: Full: every node 0 or 2 children. Complete: all levels filled except possibly last, filled left to right — heap requirement. Perfect: all internal nodes have 2 children, all leaves same depth — rare. Complete trees map cleanly to array heap indexing — know formulas for interview heap problems.

Q36: What is the degree of a vertex in a graph?

Answer: Undirected: number of edges incident to vertex. Directed: in-degree (incoming edges), out-degree (outgoing). Handshaking lemma: sum of degrees = $2E$ in undirected graph. High-degree nodes (hubs) affect BFS queue size and algorithm choice — important in social graph questions.

Q37: Why do self-balancing trees exist?

Answer: Plain BST degrades to linked list $O(n)$ on sorted input. AVL trees keep height difference ≤ 1 ; Red-Black trees relax balance for fewer rotations on insert. Both guarantee $O(\log n)$ operations. Java TreeMap uses Red-Black. Interview: you rarely implement rotations but must know unbalanced BST is dangerous.

Q38: Explain cache locality and why arrays beat linked lists in practice.

Answer: CPUs load memory in cache lines (~ 64 bytes). Array elements contiguous — prefetch hits cache. Linked list nodes scattered — pointer chasing causes cache misses. Same $O(n)$ scan but arrays often $5\text{--}10\times$ faster in real benchmarks. Mention when interviewer asks array vs linked list beyond Big-O.

Q39: How do you delete a node in a doubly linked list given a pointer?

Answer: Update `prev.next = next` and `next.prev = prev`. Handle head and tail edge cases. $O(1)$ time — why LRU uses doubly linked. Singly linked requires $O(n)$ to find predecessor unless copying next node's data (hack, fails if next is null).

Q40: What is a monotonic stack?

Answer: Stack maintaining increasing or decreasing order. Pop smaller elements before push when finding next greater element. Used in daily temperatures, largest rectangle in histogram, stock span. Each element pushed and popped once — $O(n)$ total. Pattern recognition question at Google.

Q41: What is a binary search tree delete case breakdown?

Answer: Three cases: (1) leaf — remove. (2) one child — replace with child. (3) two children — replace value with in-order successor (or predecessor), delete successor. Case 3 preserves BST property. $O(h)$ time. Practice all cases — interviewers love case 3.

Q42: How is a heap stored in an array?

Answer: Index 0 = root. For node i : left child = $2i+1$, right = $2i+2$, parent = $\text{floor}((i-1)/2)$. No pointers — cache friendly. Last non-leaf at index $\text{floor}(n/2)-1$. Heapify from there builds heap in $O(n)$. Explain this mapping when implementing heap from scratch.

Q43: What is graph edge vs vertex count notation?

Answer: V = vertices (nodes), E = edges. Sparse: $E \approx O(V)$. Dense: $E \approx O(V^2)$. BFS/DFS time $O(V+E)$. Space adjacency list $O(V+E)$. Many graph problems state constraints — $V \leq 10^5$, $E \leq 10^6$ determines if $O(V^2)$ matrix fits in time limit.

Q44: What is a sentinel node (dummy head) in linked lists?

Answer: Dummy node before real head simplifies insert/delete at head without special cases. `dummy.next = head`. Return `dummy.next` as new head after operations. Reduces off-by-one bugs in reverse, merge two sorted lists, remove nth from end. Standard interview technique — mention proactively.

Q45: Compare stack vs recursion for DFS.

Answer: Recursion uses call stack implicitly — clean code, risk stack overflow on deep graphs (10^5 depth). Iterative DFS uses explicit stack — control depth, can mark visited on stack. Tree DFS depth often safe; graph DFS on adjacency list may need iterative or increase stack limit. Tail recursion optimization limited in JavaScript.

Q46: What is a suffix tree vs trie?

Answer: Trie stores prefixes from root forward. Suffix tree stores all suffixes of a string — powerful for substring search $O(m)$ but complex to build. Suffix array + LCP is alternative. For interviews, trie suffices for autocomplete; mention suffix structures for advanced string matching (Google search index depth).

Q47: How do you merge two sorted linked lists?

Answer: Dummy head, pointer `curr`. Compare `l1` and `l2` heads, attach smaller, advance. $O(n+m)$ time, $O(1)$ space. Extension: merge K lists with min-heap — $O(N \log k)$. Microsoft favorite — tests pointer discipline without complex algorithms.

Q48: What is the Josephus problem and which structure helps?

Answer: n people in circle, every k -th eliminated — find survivor. Simulated with circular linked list $O(nk)$ or mathematical recurrence $O(n)$. Shows creative use of circular structure. Occasional math-heavy interview variant — know brute force with queue or list.

Q49: Summarize time complexities for all major structures.

Answer: Array access $O(1)$, search $O(n)$. Hash map average $O(1)$ ops. Stack/queue $O(1)$ push/pop. Linked list search $O(n)$, insert at known node $O(1)$. Balanced BST $O(\log n)$ search/insert. Heap $O(\log n)$ insert/extract. Trie $O(m)$ per string length m . Graph BFS/DFS $O(V+E)$. Recite this table fluently — baseline for every DSA round.

Q50: What is the most common mistake candidates make with linked lists?

Answer: Losing reference to head, not handling null next, creating cycles accidentally, or using `array.shift()` thinking it is $O(1)$. Always draw pointers, use dummy head, check `while (curr !== null)` vs `while (curr.next !== null)` depending on operation. Test empty list and single node — interviewers always include these edge cases.